LAB # 8

To Explain and Reproduce the Working of 7- Segment on STM32F407 Trainer Board using STM32CubeIDE and HAL Library

Objectives

- To explain the working of 7-segment interfacing with the STM32F407 using trainer board.
- To modify the program using HAL library functions and reproduce its output on STM32F407 trainer board.

Pre-Lab Exercise

Read the details given below in order to comprehend the basic operation of seven-segment display, its types and interfacing of seven-segment with microcontroller. It can be interfaced to do different tasks in different combinations.

Introduction

Seven-segment displays are now widely used in almost all microprocessor-based devices. A seven-segment display can display the digits from 0 to 9 and the hex digits A to F. Display is composed of seven LEDs that are arranged in a way to allow the display of different digits using different combinations of LEDs.

There are two types of seven-segment displays, common anode and common cathode as shown in *Figure 8-1*. In common anode, all the anodes of the LEDs are connected together in a common point. By connecting this point to the supply, we can activate the display. The data bits are connected to the cathodes of the LEDs. So, if one of the bits is low; current will flow through the corresponding LED and turn it ON. For the common cathode display, the common cathode is connected to the GND and the data bits are connected to the anodes of the LEDs.

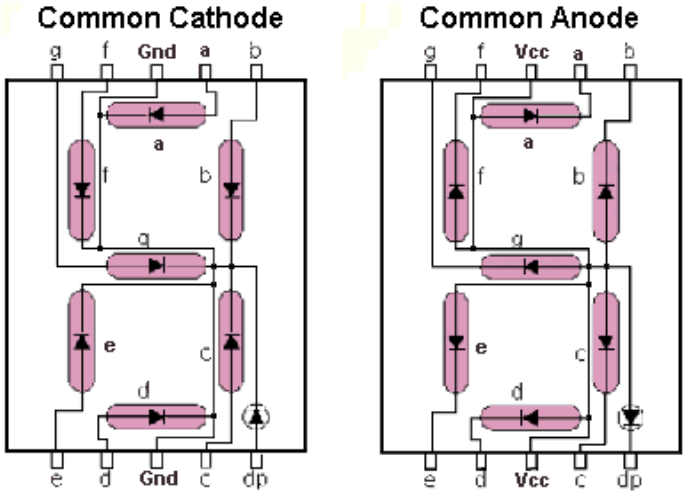


Figure 8-1: Types of 7-Segment displays

Depending upon the decimal digit to be displayed, the particular set of LEDs is forward biased. For instance, to display the numerical digit 0, we will need to light up six of the LED segments corresponding to a, b, c, d, e and f. Then the various digits from 0 through 9 can be displayed using a 7-segment display as shown in *Figure 8-2*.

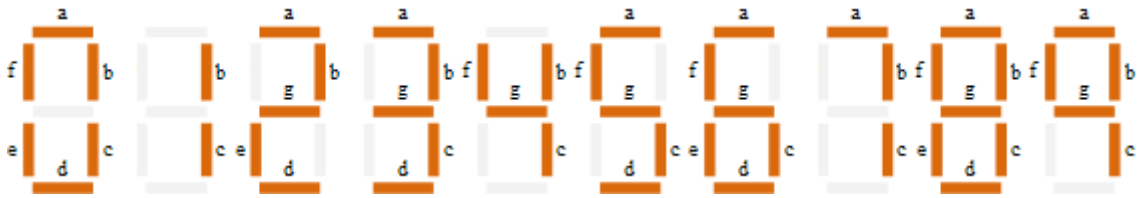


Figure 8-2: 7-Segment Display Segments for all Numbers

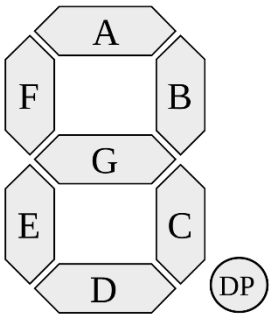
Multiplexing

Single seven segment display can display only one digit but what if we want to display more than one digit. There is a solution named Multiplexing. Consider a panel with 3 displays and number to be displayed is 123. Without using multiplexing, we need at least 3x7=21 bits for the 3 displays. Multiplexing means that only one display will have current flowing through it at one time. By activating one display at a time, we can display 3 digits turn by turn on their corresponding 7 segment display. If the multiplexing frequency is high enough, our eyes will not be able to detect the switching in the displays and they will seem to be active at the same time. Using this technique same data bits will be shared by all displays.

In-Lab Exercise

Task 1: 7 segment interfacing

Complete the program that will display the numbers displayed on 7-Segment display.



7-Segment	
a	GPIOE_PIN2
b	GPIOE_PIN5
c	GPIOC_PIN8
d	GPIOE_PIN6
e	GPIOD_PIN0
f	GPIOC_PIN9
g	GPIOE_PIN4
dot	GPIOC_PIN13
dig_1	GPIOB_PIN5
dig_2	GPIOB_PIN4
dig_3	GPIOB_PIN7

```
#include "main.h"
#include "gpio.h"
// Segment pins (active HIGH for common cathode)
#define SEG_A   GPIO_PIN_2
#define SEG_B   GPIO_PIN_5
#define SEG_C   GPIO_PIN_8
#define SEG_D   GPIO_PIN_6
#define SEG_E   GPIO_PIN_0
#define SEG_F   GPIO_PIN_9
```

```
#define SEG_G   GPIO_PIN_4
#define SEG_DP  GPIO_PIN_13

// Digit select pins
#define DIG1    GPIO_PIN_5
#define DIG2    GPIO_PIN_4
#define DIG3    GPIO_PIN_7

void Clear_Segments(void) {
    HAL_GPIO_WritePin(GPIOE, SEG_A|SEG_B|SEG_D|SEG_G, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOC, SEG_C|SEG_F, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOD, SEG_D|SEG_E, GPIO_PIN_SET);
}

void Display_Char(int ch) {
    Clear_Segments();

    switch(ch) {
        case 0:
            HAL_GPIO_WritePin(GPIOE, SEG_A, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOE, SEG_B, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOC, SEG_C, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOE, SEG_D, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOD, SEG_E, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOC, SEG_F, GPIO_PIN_RESET);

        break;
        case 1:
            HAL_GPIO_WritePin(GPIOE, SEG_B, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOC, SEG_C, GPIO_PIN_RESET);

        break;
        case 2:
            HAL_GPIO_WritePin(GPIOE, SEG_A, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOE, SEG_B, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOE, SEG_G, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOE, SEG_D, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOD, SEG_E, GPIO_PIN_RESET);
// complete the remaining cases 3 to 9
        default: break;
    }
}

void Activate_Digit(uint8_t digit) {
    HAL_GPIO_WritePin(GPIOB,  DIG1|DIG2|DIG3,  GPIO_PIN_SET);  //  Turn  off  all
digits
    switch(digit) {
        case 1: HAL_GPIO_WritePin(GPIOB, DIG1, GPIO_PIN_RESET); break;
        case 2: HAL_GPIO_WritePin(GPIOB, DIG2, GPIO_PIN_RESET); break;
        case 3: HAL_GPIO_WritePin(GPIOB, DIG3, GPIO_PIN_RESET); break;
    }
}
```

```
void SystemClock_Config(void);

int main(void)
{
  HAL_Init();
      Clear_Segments();
  SystemClock_Config();
  MX_GPIO_Init();
  while (1)
  {
    /* USER CODE END WHILE */
        for (int i=1; i<=3; i++) {
                  Activate_Digit(i);
                  Display_Char(i-1);
                  HAL_Delay(5);  // small delay for persistence of vision
        }
  }
}
```

Task 2: Write a C program to interface Up/Down counter

In this exercise we will interface 7-Segment display and two push buttons PB1 and PB2 to STM32F4 microcontroller. 7-Segment display will be used to display a 3 digit number. When push button PB1 is activated; number will be incremented by one and this incremented value will be displayed on 7-Segment display. When push button PB2 is activated; number will be start decrementing by one and this decremented value will be displayed on 7-Segment display. When both the push buttons are pressed simultaneously then the counter will reset to ZERO.

Rubric for Lab Assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all assigned tasks independently, strictly followed all health and safety protocols, and presented the results clearly and accurately.	4	
Good	The student completed the assigned tasks with minimal guidance, mostly followed health and safety protocols, and presented the results appropriately.	3	
Average	The student partially completed the assigned tasks, inconsistently followed health and safety protocols, and presented partial or unclear results.	2	
Worst	The student did not complete the assigned tasks, frequently disregarded health and safety protocols, and failed to present results.	1	

Instructor Signature: _____ Date: _____